

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6
«ВВЕДЕНИЕ В СУБД MONGODB. УСТАНОВКА MONGODB.
НАЧАЛО РАБОТЫ С БД» по дисциплине «Проектирование и
реализация баз данных»**

Обучающийся Левчук Софья Александровна
Факультет прикладной информатики
Группа К3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД MongoDB 8.2.

2 CRUD-ОПЕРАЦИИ В СУБД MONGODB. ВСТАВКА ДАННЫХ. ВЫБОРКА ДАННЫХ

2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

Практическое задание 2.1.1:

1) *Создайте базу данных learn.*

2) *Заполните коллекцию единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

3) *Используя второй способ, вставьте в коллекцию единорогов документ:*

```
{name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

4) *Проверьте содержимое коллекции с помощью метода find.*

Выполнение:

1) Создаем базу данных learn

```
test> use learn
switched to db learn
learn> |
```

2) Заполняем коллекцию unicorns (создается автоматически):

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

3) Используя второй способ, добавляем еще одного единорога:

```
var doc = {name: 'Dunx', loves: ['grape','watermelon'], weight: 704, gender: 'm',  
vampires: 165};
```

```
db.unicorns.insert(doc);
```

4) Проверяем:

```
db.unicorns.find()
```

```
learn> db.unicorns.find()
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
]
```

```
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc118'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc119'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc11a'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc11b'),
  name: 'Raleigh',
  loves: [ 'apple', 'sugar' ],
  weight: 421,
  gender: 'm',
  vampires: 2
},
]
```

```

{
  _id: ObjectId('6a06e0aaf5e15d67a1abc11c'),
  name: 'Leia',
  loves: [ 'apple', 'watermelon' ],
  weight: 601,
  gender: 'f',
  vampires: 33
},
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc11d'),
  name: 'Pilot',
  loves: [ 'apple', 'watermelon' ],
  weight: 650,
  gender: 'm',
  vampires: 54
},
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc11e'),
  name: 'Nimue',
  loves: [ 'grape', 'carrot' ],
  weight: 540,
  gender: 'f'
},
{
  _id: ObjectId('6a06e1c0f5e15d67a1abc120'),
  name: 'Dunx',
  loves: [ 'grape', 'watermelon' ],
  weight: 704,
  gender: 'm',
  vampires: 165
}
]
learn>

```

2.2 ВЫБОРКА ДАННЫХ ИЗ БД

Практическое задание 2.2.1:

1) Сформируйте запросы для вывода списков самцов и самок единорогов. Ограничьте список самок первыми тремя особями. Отсортируйте списки по имени.

2) Найдите всех самок, которые любят carrot. Ограничьте этот список первой особью с помощью функций *findOne* и *limit*.

Выполнение:

- 1) Список самцов с сортировкой по имени и без ограничений:
`db.unicorns.find({gender: 'm'}).sort({name: 1})`

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a06e1c0f5e15d67a1abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc114'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11a'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc117'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc116'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
learn> |
```

Список первых трех самок с сортировкой по имени:
 db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc119'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11c'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
learn> |
```

- 2) Список самок, любящих морковь (1 особь)
 findOne: db.unicorns.findOne({gender: 'f', loves: 'carrot'})

```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc115'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
}
learn> |
```

limit: db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)

```
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> |
```

Практическое задание 2.2.2:

Модифицируйте запрос для вывода списков самцов единорогов, исключив из результата информацию о предпочтениях и поле.

Выполнение:

db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0})
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc114'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc117'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11b'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11d'),
    name: 'Pilot',
    weight: 650,
    vampires: 54
  },
  {
    _id: ObjectId('6a06e1c0f5e15d67a1abc120'),
    name: 'Dunx',
    weight: 704,
    vampires: 165
  }
]
learn> |
```

Практическое задание 2.2.3:

Вывести список единорогов в обратном порядке добавления.

Выполнение:

`db.unicorns.find().sort({$natural: -1})`

(на скриншоте часть вывода)

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a06e1c0f5e15d67a1abc120'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11c'),
    name: 'Rory',
    loves: [ 'apple', 'watermelon' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11b'),
    name: 'Raleigh',
    weight: 421,
    vampires: 2
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11a'),
    name: 'Kenny',
    weight: 690,
    vampires: 39
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc119'),
    name: 'Rory',
    loves: [ 'apple', 'watermelon' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc118'),
    name: 'Rory',
    loves: [ 'apple', 'watermelon' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc117'),
    name: 'Rooooooodles',
    weight: 575,
    vampires: 99
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc116'),
    name: 'Unicrom',
    weight: 984,
    vampires: 182
  },
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc115'),
    name: 'Horny',
    weight: 600,
    vampires: 63
  }
]
```

Практическое задание 2.2.4:

Вывести список единорогов с названием первого любимого предпочтения, исключив идентификатор.

Выполнение:

```
db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
```

```
learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]
learn>
```

2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Практическое задание 2.3.1:

Вывести список самок единорогов весом от полутонны до 700 кг, исключив вывод идентификатора.

Выполнение:

```
db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
```

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Практическое задание 2.3.2:

Вывести список самцов единорогов весом от полутонны и предпочитающих grape и lemon, исключив вывод идентификатора.

Выполнение:

```
db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape','lemon']}}, {_id: 0})
```



```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}, {_id: 0}})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
learn> |
```

Практическое задание 2.3.3:

Найти всех единорогов, не имеющих ключ vampires.

Выполнение:

db.unicorns.find({vampires: {\$exists: false}})

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11e'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
learn> |
```

Практическое задание 2.3.4:

Вывести список упорядоченный список имен самцов единорогов с информацией об их первом предпочтении.

Выполнение:

db.unicorns.find({gender: 'm'}, {name: 1, loves: {\$slice: 1}, _id: 0}).sort({name: 1})

```
learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
learn> |
```

3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB.

ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1:

1) Создайте коллекцию towns, включающую следующие документы:

```
{name: "Punxsutawney ",
  populatiuon: 6200,
  last_sensus: ISODate("2008-01-31"),
```

```
famous_for: [""],
mayor: {
  name: "Jim Wehrle"
}}
```

```
{name: "New York",
populatiuon: 22200000,
last_sensus: ISODate("2009-07-31"),
famous_for: ["status of liberty", "food"],
mayor: {
  name: "Michael Bloomberg",
  party: "I"}}
```

```
{name: "Portland",
populatiuon: 528000,
last_sensus: ISODate("2009-07-20"),
famous_for: ["beer", "food"],
mayor: {
  name: "Sam Adams",
  party: "D"}}
```

- 2) Сформировать запрос, который возвращает список городов с независимыми мэрами (party="I"). Вывести только название города и информацию о мэре.
- 3) Сформировать запрос, который возвращает список беспартийных мэров (party отсутствует). Вывести только название города и информацию о мэре.

Выполнение:

- 1) Заполняем коллекцию (я также исправила опечатку):

```
db.towns.insert({
  name: "Punxsutawney",
  population: 6200,
  last_sensus: ISODate("2008-01-31"),
  famous_for: ["phil the groundhog"],
  mayor: { name: "Jim Wehrle" }
});
```

```
db.towns.insert({
  name: "New York",
  population: 22200000,
  last_sensus: ISODate("2009-07-31"),
  famous_for: ["statue of liberty", "food"],
  mayor: { name: "Michael Bloomberg", party: "I" }
});
```

```
db.towns.insert({
  name: "Portland",
  population: 528000,
  last_sensus: ISODate("2009-07-20"),
  famous_for: ["beer", "food"],
  mayor: { name: "Sam Adams", party: "D" }
});
```

- 2) Список городов с независимыми мэрами (название города + информация о мэре):

```
db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
learn> |
```

Список беспартийных мэров (название города + информация о мэре):

```
db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
```

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
learn> |
```

Практическое задание 3.1.2:

- 1) *Сформировать функцию для вывода списка самцов единорогов.*
- 2) *Создать курсор для этого списка из первых двух особей с сортировкой в лексикографическом порядке.*
- 3) *Вывести результат, используя forEach.*
- 4) *Содержание коллекции единорогов unicorns:*

```
db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
```

```
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
```

```
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
```

```
db.unicorns.insert({name: 'Roooooodles', 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
```

```
db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
```

```
db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
```

```
db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
```

```
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
```

```
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
```

```
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
```

```
db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
```

```
db.unicorns.insert({name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165});
```

Выполнение:

- 1) Функция для вывода самцов:


```
function maleUnicorns() {
  return db.unicorns.find({gender: 'm'});
}
```
- 2) Курсор для данного списка из первых двух особей с сортировкой по алфавиту:


```
var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2);
```
- 3) Вывод результата с использованием forEach:


```
cursor.forEach(function(u) { print(u.name, u.weight); });
```

```
learn> cursor.forEach(function(u) { print(u.name, u.weight); });
Dunx 704
Horny 600
```

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Практическое задание 3.2.1:

Вывести количество самок единорогов весом от полутонны до 600 кг.

Выполнение:

```
db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
```

```
learn> db.unicorns.countDocuments({gender: 'f', weight: {$gte: 500, $lte: 600}})
2
learn> |
```

Практическое задание 3.2.2:

Вывести список предпочтений.

Выполнение:

```
db.unicorns.distinct("loves")
```

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
learn> |
```

Практическое задание 3.2.3:

Посчитать количество особей единорогов обоих полов.

Выполнение:

```
db.unicorns.aggregate([
  {$group: {_id: "$gender", count: {$sum: 1}}}
])
```

```
learn> db.unicorns.aggregate([
|   {$group: {_id: "$gender", count: {$sum: 1}}}
| ])
[ { _id: 'f', count: 5 }, { _id: 'm', count: 7 } ]
learn>
```

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

Практическое задание 3.3.1:

1. *Выполнить команду:*

```
> db.unicorns.save({name: 'Barny', loves: ['grape'],
weight: 340, gender: 'm'})
```

2. *Проверить содержимое коллекции unicorns.*

Выполнение:

- 1) При попытке выполнить команду из задания я столкнулась с ошибкой:

```
learn> db.unicorns.save({name: 'Barney', loves: ['grape'],
| weight: 340, gender: 'm'})
|
TypeError: db.unicorns.save is not a function
```

Она связана с тем, что save устарела и удалена в новых версиях MongoDB. Поэтому я использовала insertOne:

db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340, gender: 'm'})

```
learn> db.unicorns.insertOne({name: 'Barney', loves: ['grape'], weight: 340,
gender: 'm'})
{
  acknowledged: true,
  insertedId: ObjectId('6a070611f5e15d67a1abc124')
}
learn>
```

- 2) Проверим содержимое и увидим, что появился новый единорог:

```
{
  _id: ObjectId('6a070611f5e15d67a1abc124'),
  name: 'Barney',
  loves: [ 'grape' ],
  weight: 340,
  gender: 'm'
}
]
learn> |
```

Практическое задание 3.3.2:

1. Для самки единорога Айна внести изменения в БД: теперь ее вес 800, она убила 51 вампира.
2. Проверить содержимое коллекции unicorns.

Выполнение:

- 1) db.unicorns.update({name: 'Ayna'}, {\$set: {weight: 800, vampires: 51}})
- 2) Проверим данные конкретно этого единорога:

db.unicorns.find({name: 'Ayna'})

```
learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc119')
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
learn>
```

Практическое задание 3.3.3:

1. Для самца единорога Raleigh внести изменения в БД: теперь он любит рэдбул.
2. Проверить содержимое коллекции unicorns.

Выполнение:

- 1) db.unicorns.update({name: 'Raleigh'}, {\$push: {loves: 'redbull'}})
- 2) db.unicorns.find({name: 'Raleigh'})

```
learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11b'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
learn>
```

Практическое задание 3.3.4:

1. *Всем самцам единорогов увеличить количество убитых вампиров на 5.*
2. *Проверить содержимое коллекции unicorns.*

Выполнение:

Зафиксируем одного единорога в начале для примера:

```
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc114'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
},
```

- 1) db.unicorns.updateMany({gender: 'm'}, {\$inc: {vampires: 5}})
- 2) db.unicorns.findOne({name: 'Horny'})

```
learn> db.unicorns.findOne({name: 'Horny'})
{
  _id: ObjectId('6a06e0aaf5e15d67a1abc114'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 68
}
learn>
```

Практическое задание 3.3.5:

1. *Изменить информацию о городе Портланд: мэр этого города теперь беспартийный.*
2. *Проверить содержимое коллекции towns.*

Выполнение:

- 1) db.towns.updateOne(
 { name: "Portland" },
 { \$unset: { "mayor.party": "" } }
)
- 2) db.towns.find({name: 'Portland'})

```
learn> db.towns.find({name: 'Portland'})
[
  {
    _id: ObjectId('6a06fafaf5e15d67a1abc123'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
learn> |
```

Практическое задание 3.3.6:

1. Изменить информацию о самце единорога Pilot: теперь он любит и шоколад.
2. Проверить содержимое коллекции unicorns.

Выполнение:

- 1) db.unicorns.update({name: 'Pilot'}, {\$push: {loves: 'chocolate'}})
- 2) db.unicorns.find({name: 'Pilot'})

```
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc11d'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
learn> |
```

Практическое задание 3.3.7:

1. Изменить информацию о самке единорога Aurora: теперь она любит еще и сахар, и лимоны.
2. Проверить содержимое коллекции unicorns.

Выполнение:

- 1) db.unicorns.update({name: 'Aurora'}, {\$addToSet: {loves: {\$each: ['sugar','lemon']}}})
- 2) db.unicorns.find({name: 'Aurora'})

```
learn> db.unicorns.find({name: 'Aurora'})
[
  {
    _id: ObjectId('6a06e0aaf5e15d67a1abc115'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn>
```

3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

Практическое задание 3.4.1:

1) *Создайте коллекцию towns, включающую следующие документы:*

```
{name: "Punxsutawney ",  
population: 6200,  
last_sensus: ISODate("2008-01-31"),  
famous_for: ["phil the groundhog"],  
mayor: {  
  name: "Jim Wehrle"  
}}
```

```
{name: "New York",  
population: 22200000,  
last_sensus: ISODate("2009-07-31"),  
famous_for: ["statue of liberty", "food"],  
mayor: {  
  name: "Michael Bloomberg",  
  party: "I"}}}
```

```
{name: "Portland",  
population: 528000,  
last_sensus: ISODate("2009-07-20"),  
famous_for: ["beer", "food"],  
mayor: {  
  name: "Sam Adams",  
  party: "D"}}}
```

2) *Удалите документы с беспартийными мэрами.*

3) *Проверьте содержание коллекции.*

4) *Очистите коллекцию.*

5) *Просмотрите список доступных коллекций.*

Выполнение:

1) Создаем коллекцию (решила сделать новую - towns1, потому что у нас уже есть towns, с которой мы работали - на всякий случай):

```
db.towns1.insertMany([  
  {  
    name: "Punxsutawney",  
    population: 6200,  
    last_sensus: ISODate("2008-01-31"),  
    famous_for: ["phil the groundhog"],  
    mayor: { name: "Jim Wehrle" }  
  },  
  {  
    name: "New York",  
    population: 22200000,  
    last_sensus: ISODate("2009-07-31"),  
    famous_for: ["statue of liberty", "food"],  
    mayor: { name: "Michael Bloomberg", party: "I" }  
  },  
  {  
    name: "Portland",  
    population: 528000,  
    last_sensus: ISODate("2009-07-20"),
```



```
famous_for: ["beer", "food"],
mayor: { name: "Sam Adams", party: "D" }
}
]);
```

- 2) Удаляем документы с беспартийными мэрами:

```
db.towns1.deleteMany({ "mayor.party": { $exists: false } });
```

```
learn> db.towns1.deleteMany({ "mayor.party": { $exists: false } });
{ acknowledged: true, deletedCount: 1 }
learn>
```

- 3) Проверим содержимое:

```
db.towns1.find().pretty()
```

```
learn> db.towns1.find()
[
  {
    _id: ObjectId('6a070b95f5e15d67a1abc126'),
    name: 'New York',
    population: 22200000,
    last_sensus: ISODate('2009-07-31T00:00:00.000Z'),
    famous_for: [ 'statue of liberty', 'food' ],
    mayor: { name: 'Michael Bloomberg', party: 'I' }
  },
  {
    _id: ObjectId('6a070b95f5e15d67a1abc127'),
    name: 'Portland',
    population: 528000,
    last_sensus: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams', party: 'D' }
  }
]
learn>
```

- 4) Очищаем коллекцию:

```
db.towns1.deleteMany({});
```

```
learn> db.towns1.deleteMany({});
{ acknowledged: true, deletedCount: 2 }
learn>
```

- 5) Посмотрим список доступных коллекций:

```
show collections
```

```
learn> show collections
towns
towns1
unicorns
learn> |
```

4 ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

4.1 ССЫЛКИ В БД

Практическое задание 4.1.1:

- 1) Создайте коллекцию зон обитания единорогов, указав в качестве идентификатора кратко название зоны, далее включив полное название и описание.

- 2) Включите для нескольких единорогов в документы ссылку на зону обитания, используя второй способ автоматического связывания.
- 3) Проверьте содержание коллекции единорогов.
- 4) Содержание коллекции единорогов unicorns:

```
db.unicorns.insert({name: 'Horny', loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});
db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
db.unicorns.insert({name: 'Rooooooodles', 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
db.unicorns.insert({name: 'Solnara', loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});
db.unicorns.insert({name:'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
db.unicorns.insert({name:'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
db.unicorns.insert ({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
db.unicorns.insert {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
```

Выполнение:

- 1) Создадим какие-нибудь зоны обитания (взяты из Minecraft):

```
db.habitats.insertMany([
  { _id: "cherry", name: "Cherry Grove", description: "Pink petals, peaceful bees, shallow caves beneath" },
  { _id: "flower_forest", name: "Flower Forest", description: "Thousands of flowers, rabbits, and birch trees" },
  { _id: "lush_caves", name: "Lush Caves", description: "Glow berries, dripstone, axolotls, and clay pools" },
  { _id: "meadow", name: "Meadow", description: "High-altitude flower field, sheep, and beehives on birch trees" }
]);
```

```
learn> db.habitats.insertMany([
|   { _id: "cherry", name: "Cherry Grove", description: "Pink petals, peaceful bees, shallow caves beneath" },
|   { _id: "flower_forest", name: "Flower Forest", description: "Thousands of flowers, rabbits, and birch trees" },
|   { _id: "lush_caves", name: "Lush Caves", description: "Glow berries, dripstone, axolotls, and clay pools" },
|   { _id: "meadow", name: "Meadow", description: "High-altitude flower field, sheep, and beehives on birch trees" }
| ]);
| {
|   acknowledged: true,
|   insertedIds: {
|     '0': 'cherry',
|     '1': 'flower_forest',
|     '2': 'lush_caves',
|     '3': 'meadow'
|   }
| }
learn>
```

- 2) Сделаем для 4 единорогов привязку к созданным зонам:


```
db.unicorns.updateOne({ name: "Aurora" }, { $set: { habitat: { $ref: "habitats", $id: "cherry" } } });
```

```

db.unicorns.updateOne({ name: "Nimue" }, { $set: { habitat: { $ref: "habitats", $id: "flower_forest" } } });
db.unicorns.updateOne({ name: "Pilot" }, { $set: { habitat: { $ref: "habitats", $id: "meadow" } } });
db.unicorns.updateOne({ name: "Leia" }, { $set: { habitat: { $ref: "habitats", $id: "lush_caves" } } });

```

3) Проверим:

```

db.unicorns.find({ habitat: { $exists: true } }, { name: 1, habitat: 1, _id: 0 })

```

```

learn> db.unicorns.find({ habitat: { $exists: true } }, { name: 1, habitat: 1, _id: 0 })
[
  { name: 'Aurora', habitat: DBRef('habitats', 'cherry') },
  { name: 'Leia', habitat: DBRef('habitats', 'lush_caves') },
  { name: 'Pilot', habitat: DBRef('habitats', 'meadow') },
  { name: 'Nimue', habitat: DBRef('habitats', 'flower_forest') }
]
learn>

```

4.2 НАСТРОЙКА ИНДЕКСОВ

Практическое задание 4.2.1:

1. Проверьте, можно ли задать для коллекции *unicorns* индекс для ключа *name* с флагом *unique*.
2. Содержание коллекции единорогов *unicorns*:

```

db.unicorns.insert({name: 'Horny', dob: new Date(1992,2,13,7,47), loves: ['carrot','papaya'], weight: 600, gender: 'm', vampires: 63});

```

```

db.unicorns.insert({name: 'Aurora', dob: new Date(1991, 0, 24, 13, 0), loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});

```

```

db.unicorns.insert({name: 'Unicrom', dob: new Date(1973, 1, 9, 22, 10), loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});

```

```

db.unicorns.insert({name: 'Roooooodles', dob: new Date(1979, 7, 18, 18, 44), loves: ['apple'], weight: 575, gender: 'm', vampires: 99});

```

```

db.unicorns.insert({name: 'Solnara', dob: new Date(1985, 6, 4, 2, 1), loves:['apple', 'carrot', 'chocolate'], weight:550, gender:'f', vampires:80});

```

```

db.unicorns.insert({name:'Ayna', dob: new Date(1998, 2, 7, 8, 30), loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});

```

```

db.unicorns.insert({name:'Kenny', dob: new Date(1997, 6, 1, 10, 42), loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});

```

```

db.unicorns.insert({name: 'Raleigh', dob: new Date(2005, 4, 3, 0, 57), loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});

```

```

db.unicorns.insert({name: 'Leia', dob: new Date(2001, 9, 8, 14, 53), loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});

```

```

db.unicorns.insert({name: 'Pilot', dob: new Date(1997, 2, 1, 5, 3), loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});

```

```

db.unicorns.insert ({name: 'Nimue', dob: new Date(1999, 11, 20, 16, 15), loves: ['grape', 'carrot'], weight: 540, gender: 'f'});

```

```
db.unicorns.insert ({name: 'Dunx', dob: new Date(1976, 6, 18, 18, 18), loves: ['grape', 'watermelon'],  
weight: 704, gender: 'm', vampires: 165});
```

Выполнение:

- 1) Сначала проверим, есть ли дубликаты имен:

```
db.unicorns.aggregate([  
  { $group: { _id: "$name", count: { $sum: 1 } } },  
  { $match: { count: { $gt: 1 } } }  
]);
```

```
learn> db.unicorns.aggregate([  
|   { $group: { _id: "$name", count: { $sum: 1 } } },  
|   { $match: { count: { $gt: 1 } } }  
| ]]);  
learn>
```

Пустая строка - пустой массив, следовательно, дубликатов нет

- 2) Создаем уникальный индекс по имени:

```
db.unicorns.createIndex({ name: 1 }, { unique: true });
```

```
learn> db.unicorns.createIndex({ name: 1 }, { unique: true });  
name_1  
learn>
```

- 3) Попробуем добавить единорога с уже существующим именем:

```
db.unicorns.insertOne({ name: "Horny", loves: ["sugar"], weight: 500, gender: "m" });
```

```
learn> db.unicorns.insertOne({ name: "Horny", loves: ["sugar"], weight: 500, gender: "m" });  
MongoServerError: E11000 duplicate key error collection: learn.unicorns index: name_1 dup key: { name: "Horny" }  
learn>
```

4.3 УПРАВЛЕНИЕ ИНДЕКСАМИ

Практическое задание 4.3.1:

- 1) Получите информацию о всех индексах коллекции unicorns.
- 2) Удалите все индексы, кроме индекса для идентификатора.
- 3) Попытайтесь удалить индекс для идентификатора.

Выполнение:

- 1) Посмотрим все индексы коллекции unicorns:

```
db.unicorns.getIndexes();  
  
learn> db.unicorns.getIndexes();  
[  
  { v: 2, key: { _id: 1 }, name: '_id_', },  
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }  
]  
learn>
```

- 2) Удаляем все индексы кроме идентификатора:

```
db.unicorns.dropIndex("name_1");  
  
learn> db.unicorns.dropIndex("name_1");  
{ nIndexesWas: 2, ok: 1 }  
learn>
```

- 3) Попробуем удалить индекс для идентификатора:

```
db.unicorns.dropIndex("_id");  
  
learn> db.unicorns.dropIndex("_id");  
MongoServerError[InvalidOptions]: cannot drop _id index  
learn>
```

4.4 ПЛАН ЗАПРОСА

Практическое задание 4.4.1:

- 1) Создайте объемную коллекцию *numbers*, задействовав курсор:

```
for(i = 0; i < 100000; i++){db.numbers.insert({value: i})}
```
- 2) Выберите последних четыре документа.
- 3) Проанализируйте план выполнения запроса 2. Сколько потребовалось времени на выполнение запроса? (по значению параметра *executionTimeMillis*)
- 4) Создайте индекс для ключа *value*.
- 5) Получите информацию о всех индексах коллекции *numbers*.
- 6) Выполните запрос 2.
- 7) Проанализируйте план выполнения запроса с установленным индексом. Сколько потребовалось времени на выполнение запроса?
- 8) Сравните время выполнения запросов с индексом и без. Дайте ответ на вопрос: какой запрос более эффективен?

Выполнение:

- 1) Создаем коллекцию *numbers* (выполняем команду и просто ждем завершения):

```
for (let i = 0; i < 100000; i++) { db.numbers.insertOne({ value: i }); }
```

```
learn> for (let i = 0; i < 100000; i++) { db.numbers.insertOne({ value: i }) }  
.  
.  
.  
acknowledged: true,  
insertedId: ObjectId('6a071bc3f5e15d67a1adb1c8')  
}  
learn> |
```

Можем проверить количество:

```
learn> db.numbers.countDocuments()  
100000  
learn> |
```

- 2) Выбираем последние 4 документа:

```
db.numbers.find().sort({$natural: -1}).limit(4)
```

```
learn> db.numbers.find().sort({$natural: -1}).limit(4)  
[  
  { _id: ObjectId('6a071bc3f5e15d67a1adb1c8'), value: 99999 },  
  { _id: ObjectId('6a071bc3f5e15d67a1adb1c7'), value: 99998 },  
  { _id: ObjectId('6a071bc3f5e15d67a1adb1c6'), value: 99997 },  
  { _id: ObjectId('6a071bc3f5e15d67a1adb1c5'), value: 99996 }  
]  
learn> |
```

- 3) Анализируем план выполнения запроса 2:

```
db.numbers.find().sort({$natural: -1}).limit(4).explain("executionStats")
```

Получаем большой вывод, из которого посмотрим на последний блок *executionStats*:

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 151,
  totalKeysExamined: 0,
  totalDocsExamined: 100000,
  executionStages: {
    isCached: false,
    stage: 'SORT',
    nReturned: 4,
  }
}

```

Значение executionTimeMillis: 151

- 4) Создаем индекс для ключа value:

```
db.numbers.createIndex({ value: 1 })
```

```

learn> db.numbers.createIndex({ value: 1 })
value_1
learn>

```

- 5) Посмотрим все индексы:

```
db.numbers.getIndexes()
```

```

learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_' },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
learn>

```

- 6) Снова выполним запрос 2

```
db.numbers.find().sort({ value: -1 }).limit(4).explain("executionStats")
```

```

executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 1,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
    isCached: false,
    stage: 'LIMIT',
    nReturned: 4,
  }
}

```

Значение executionTimeMillis: 1

- 7) Без индекса время выполнения запроса - 151 мс, с индексом - 1 мс. Запрос с индексом эффективнее, так как без индекса MongoDB сканирует всю коллекцию и сортирует документы, а с индексом использует направленный обход индекса, что выходит гораздо быстрее.

Вывод

В ходе выполнения лабораторной работы были освоены практические навыки работы с СУБД MongoDB:

- выполнение CRUD-операций (вставка документов методами insertOne и insertMany, чтение с фильтрацией через find с операторами сравнения и логики, обновление с помощью updateOne, updateMany и операторов \$set, \$inc, \$push, \$unset, а также удаление через deleteOne и deleteMany)
- работа с вложенными документами (доступ к полям через точку, например, mayor.party, обновление и удаление встроенных полей)

- агрегация данных (подсчет количества с `countDocuments`, получение уникальных значений через `distinct`, групповая агрегация с `$group`)
- создание ссылок между коллекциями (ручное связывание через сохранение `_id` и автоматическое с помощью `DBRef`)
- управление индексами (создание уникального индекса на поле `name`, удаление индекса, проверка невозможности удаления индекса по `_id`, анализ производительности запросов с индексом и без с использованием `explain`).